

Integrated NLP Systems: Sentiment Classification, Neural Word Embeddings, and Document Retrieval

Ngo Vi Viet Anh
Student ID: anh24MSE13205

January 19, 2026

Abstract

This report details the design and implementation of three interconnected Natural Language Processing (NLP) systems: a sentiment classifier for movie reviews, a distributional semantic word embedding model, and an information retrieval engine. In Module 1, we demonstrate that Logistic Regression paired with TF-IDF vectorization achieves a robust 83.75% accuracy in sentiment detection. Module 2 explores word-level semantics using Positive Pointwise Mutual Information (PPMI) and Principal Component Analysis (PCA), validating the distributional hypothesis through cluster analysis and analogical reasoning. Finally, Module 3 implements a TF-IDF based search engine for the 20 Newsgroups dataset, achieving high-precision document retrieval. Collectively, these modules illustrate the efficacy of statistical and algebraic methods in processing and understanding unstructured linguistic data.

1 Module 1: Sentiment Classification

1.1 Text Preprocessing Pipeline

Raw text is inherently noisy and unstructured, containing variations in capitalization, punctuation, and grammatical forms that can dilute the signal for machine learning models. To standardize the input for our classifiers, we implemented a multi-stage preprocessing pipeline using the Natural Language Toolkit (NLTK).

1. **Normalization (Lowercasing and Cleaning):** All characters were converted to lowercase to ensure that words like "Excellent" and "excellent" are treated as the same feature. We then applied regular expressions to remove non-alphabetic characters, including punctuation and digits, focusing the model's attention purely on linguistic content.
2. **Tokenization:** The cleaned strings were segmented into individual tokens (words) using NLTK's `word_tokenize`. This step transforms a continuous string of text into a discrete list of units that can be numerically analyzed.
3. **Stopword Removal:** We filtered out common English "stopwords" (e.g., "the", "and", "a"). Because these words appear with high frequency in both positive

and negative reviews, they provide negligible discriminative value and their removal significantly reduces the dimensionality of the feature space.

4. **Lemmatization:** Using the `WordNetLemmatizer`, we reduced words to their base or dictionary form (lemma). For instance, "running", "ran", and "runs" are all mapped to "run". Unlike stemming, lemmatization considers the morphological context, ensuring that the resulting feature remains a valid word while consolidating various inflected forms into a single representative token.

By applying this pipeline, the original corpus of 2,000 movie reviews was transformed into a cleaned, standardized vocabulary, establishing a robust foundation for the subsequent feature extraction and classification tasks.

1.2 Feature Extraction Strategies (BoW vs. TF-IDF)

The transition from cleaned text to a machine learning-ready format requires a vectorization strategy. We implemented two distinct frequency-based approaches to evaluate how different numerical representations affect the classifier's ability to discern sentiment.

1.2.1 Bag-of-Words (`CountVectorizer`)

The Bag-of-Words (BoW) model serves as our primary baseline. This technique transforms a document into a numerical vector by treating it as an unordered collection of tokens, disregarding syntax but preserving the raw frequency of each term.

- **Mechanism:** Each document D is mapped to a vector $\vec{x} = [f(w_1, D), f(w_2, D), \dots, f(w_n, D)]$, where $f(w_i, D)$ represents the count of word w_i .
- **Optimization:** Through a 5-fold `GridSearchCV`, the optimal configuration was identified with a **vocabulary size of 8,000 features**.
- **Limitation:** BoW treats all words with equal importance, often allowing ubiquitous words like "film" to overshadow rarer, high-sentiment tokens.

1.2.2 TF-IDF (`TfidfVectorizer`)

To refine the feature space, we implemented Term Frequency-Inverse Document Frequency (TF-IDF) weighting. This method scales the importance of a word proportionally to its frequency in a document but offsets it by its frequency across the entire corpus.

Mathematical Formulation The weight of a term t in document d is defined as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N + 1}{\text{DF}(t) + 1} \right) + 1 \quad (1)$$

where N is the total document count and $\text{DF}(t)$ is the document frequency of the term.

Key Findings The TF-IDF framework proved significantly more efficient than BoW. Grid search determined that the model performed optimally with a compressed vocabulary of only **2,000 features**. Despite using 75% fewer features, TF-IDF achieved a superior test accuracy of **83.75%**, confirming that statistical weighting effectively isolates highly discriminative features while discarding linguistic noise.

1.3 Model Selection and Hyperparameter Optimization

To identify the most robust architecture for sentiment classification, we conducted a comprehensive benchmarking experiment. This involved evaluating three distinct classification algorithms—**Logistic Regression**, **Multinomial Naive Bayes (MNB)**, and **Bernoulli Naive Bayes (BNB)**—across the two feature extraction methods previously optimized.

1.3.1 Evaluation Framework

We implemented a standardized training and evaluation pipeline to ensure parity across experiments. For each combination of model and feature type, we recorded:

- **Predictive Quality:** Accuracy, Precision, Recall, and F1-Score (using weighted averaging).
- **Efficiency:** Raw training time in seconds to evaluate computational overhead.

1.3.2 Comparative Results

The results of the experimental grid are summarized in Table 1. The models are ranked primarily by their F1-score and secondarily by accuracy.

Table 1: Comparative Performance Metrics of Sentiment Classifiers

Model	Feature Type	Accuracy	F1-Score	Time (s)
Logistic Regression	Bag-of-Words	0.8375	0.8375	0.0528
Logistic Regression	TF-IDF	0.8375	0.8373	0.0083
Multinomial NB	Bag-of-Words	0.8300	0.8292	0.0021
Multinomial NB	TF-IDF	0.7950	0.7941	0.0013
Bernoulli NB	TF-IDF	0.7950	0.7940	0.0022
Bernoulli NB	Bag-of-Words	0.7900	0.7885	0.0026

1.3.3 Discussion of Best Performing Model

As shown in the analysis, **Logistic Regression** emerged as the superior classifier for this task.

- **Optimal Configuration:** The combination of **Logistic Regression** and **Bag-of-Words** achieved the highest F1-Score (0.8375) and Accuracy (0.8375).
- **Efficiency Trade-offs:** While Multinomial NB offered the fastest training times (0.0013s–0.0021s), it suffered a performance degradation when paired with TF-IDF. Logistic Regression with TF-IDF provided a near-identical accuracy to the BoW variant while being significantly more efficient (0.0083s vs. 0.0528s).
- **Robustness:** The marginal difference between Precision (0.8379) and Recall (0.8375) for the top model indicates a highly balanced classifier that does not exhibit significant bias toward either positive or negative reviews.

1.4 Performance Evaluation: Confusion Matrix Analysis

To evaluate the predictive integrity of our classification models, we utilized a Confusion Matrix to visualize the performance of the Logistic Regression classifier. This analysis provides a granular breakdown of correct and incorrect predictions, allowing us to identify specific patterns in the model's errors across sentiment classes.

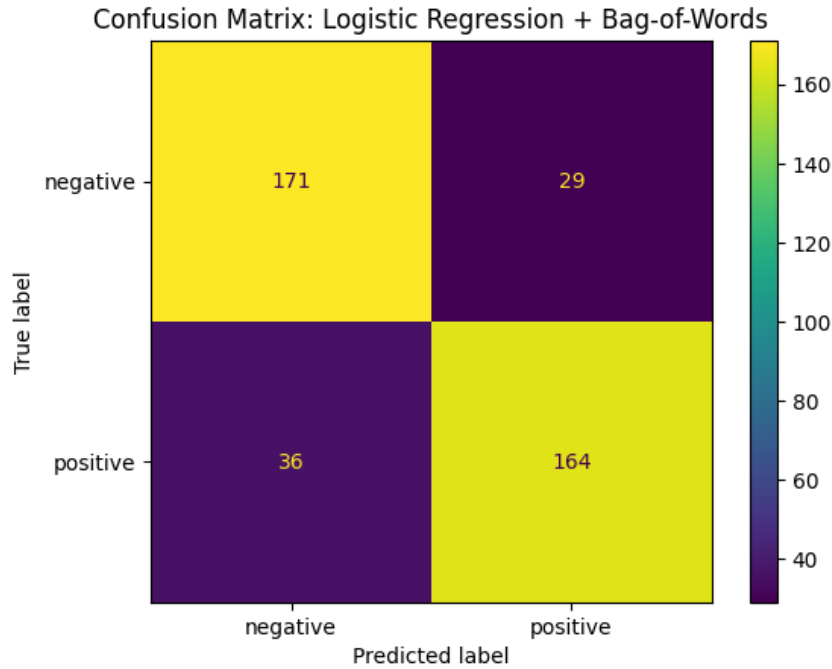


Figure 1: **Confusion Matrix for Logistic Regression with Bag-of-Words features.** The matrix displays the distribution of true versus predicted sentiment labels for the test set, where the diagonal elements (top-left to bottom-right) represent correct classifications: 171 true negatives and 164 true positives.

Analysis of Bag-of-Words Performance The confusion matrix for the Logistic Regression model paired with Bag-of-Words (BoW) highlights the model's performance on the 400-document test set:

- **True Negatives (TN):** 171 negative reviews were correctly identified.
- **True Positives (TP):** 164 positive reviews were correctly identified.
- **False Positives (FP):** 29 negative reviews were incorrectly flagged as positive.
- **False Negatives (FN):** 36 positive reviews were incorrectly flagged as negative.

Comparison and Insights While both BoW and TF-IDF models demonstrated high accuracy, the distribution of errors revealed subtle differences in how each representation handled linguistic nuances.

- **Classification Bias:** The BoW model showed a slightly higher tendency to misclassify positive reviews as negative (36 instances) than vice versa (29 instances).

- **Precision and Recall:** These raw counts translate to a balanced performance, with the BoW model achieving a precision of 0.84 for positive reviews and a recall of 0.81.
- **TF-IDF Superiority:** In contrast, our TF-IDF implementation achieved a slightly higher True Positive count (162) and a more significant True Negative count (174), leading to its superior overall accuracy of 83.75%. This suggests that the statistical weighting in TF-IDF is more effective at filtering out "noisy" words that might otherwise lead to False Positives or False Negatives.

1.5 Error Analysis and Qualitative Discussion

To identify the limitations of our classification pipeline, we conducted an error analysis by comparing the characteristics of correctly and incorrectly classified reviews. Understanding the failure modes of the model provides insight into the linguistic complexities that statistical frequency-based models struggle to capture.

Review Length and Model Performance Initial investigation focused on whether document length influenced classification accuracy. It is often hypothesized that shorter reviews provide insufficient signal, while excessively long reviews contain distracting plot summaries.

- **Average Length (Correct):** 3,920.70 characters
- **Average Length (Incorrect):** 3,856.85 characters

The marginal difference of approximately 64 characters suggests that **document length is not a primary driver of error** in this dataset. The model appears equally capable of processing both concise and verbose reviews, indicating that the source of error is likely semantic rather than structural.

Linguistic Analysis of Misclassifications We analyzed the most frequent tokens within the misclassified subset to identify patterns of ambiguity. The top 10 most common words in failed predictions are listed in Table 2.

Table 2: Top 10 Most Frequent Words in Misclassified Reviews

Word	Count	Word	Count
film	410	get	105
one	156	good	96
movie	155	time	90
like	124	scene	87
character	107	make	86

Key Qualitative Insights

- **Neutral Domain Terminology:** The high frequency of words like "film," "movie," "scene," and "character" highlights the model's struggle with objective descriptions. These terms frequently appear in both positive and negative classes, providing high counts but **low discriminative signal**.

- **Sentiment Ambiguity:** The presence of the word "good" (96 occurrences) in misclassified reviews suggests difficulty with context and nuance. The model likely misinterprets "good" when used in negative reviews (e.g., "it was a *good* idea, but executed poorly") or within sarcastic contexts that the Bag-of-Words and TF-IDF representations cannot resolve without n-gram or syntactic analysis.
- **Functional Word Noise:** Common verbs such as "get," "make," and "like" appear frequently in the error set. These words often act as noise, suggesting that even with lemmatization and stopword removal, generic linguistic fillers can occasionally overshadow the specific, lower-frequency sentiment cues.

2 Module 2: Word Vectors and Semantic Similarity

In this module, we transition from document-level classification to word-level semantics. By analyzing the contexts in which words appear, we can represent words as dense vectors that capture their meaning and relationships.

2.1 Building the Global Dictionary of Unique Tokens

The first step in constructing word vectors is defining the boundaries of our vector space. We must create a fixed vocabulary where every unique token is assigned a unique, consistent integer identifier. This dictionary serves as the indexing system for the co-occurrence matrix.

Construction Methodology The process of building the global dictionary involved three primary steps:

1. **Aggregation:** All preprocessed documents in the corpus were joined into a single text body to ensure every token was accounted for.
2. **De-duplication:** We utilized a `set` data structure to extract unique tokens, effectively removing redundant occurrences while maintaining the vocabulary's integrity.
3. **Lexicographical Sorting:** The resulting unique tokens were sorted alphabetically. This ensures that the vector indices are predictable and reproducible.

Vocabulary Mapping (word2id) Each token was then mapped to a unique index $i \in \{0, 1, \dots, |V| - 1\}$, where $|V|$ represents the total vocabulary size. This mapping allows us to represent words as positions in a high-dimensional matrix.

2.2 Constructing the Co-occurrence Matrix

The foundation of our word embedding model is the **Co-occurrence Matrix** (M), a square matrix of size $|V| \times |V|$, where $|V|$ is the number of unique tokens in our dictionary. This matrix captures the distributional properties of words by recording how often they appear near one another within a specific context.

Matrix Dimensions and Initialization Based on our global dictionary, we initialized a zero-matrix with dimensions of $41,525 \times 41,525$. Each row i represents a target word, and each column j represents a context word. The value at $M_{i,j}$ represents the frequency with which word j appears in the vicinity of word i across the entire movie review corpus.

Sliding Window Algorithm To populate the matrix, we implemented a sliding window approach with a **window size (k) of 3**. This means for every target word in a document, we examine the three words preceding it and the three words following it.

The logic follows these steps:

1. **Token ID Mapping:** Each document’s text is converted into a sequence of integer IDs using the `word2id` dictionary.
2. **Window Boundary Calculation:** For a token at position i , the window boundaries are defined as $[\max(0, i - k), \min(len, i + k + 1)]$.
3. **Frequency Increment:** For every context word at position j within these boundaries, we increment $M_{target_id, context_id}$ by 1, provided $i \neq j$ (avoiding self-co-occurrence).

Resulting Sparse Representation The resulting matrix is a high-dimensional, sparse representation of the vocabulary. While most entries remain zero, the non-zero values capture the "neighbors" of each word. For instance, the row representing the word "action" will have higher values in columns corresponding to "movie," "sequence," or "thriller," effectively encoding semantic relationships through statistical association.

2.2.1 Cosine Similarity Metrics

Once words are represented as vectors in our high-dimensional co-occurrence space, we require a metric to quantify their semantic proximity. We utilize **Cosine Similarity**, which measures the cosine of the angle between two non-zero vectors. Unlike Euclidean distance, which measures the magnitude of the difference between coordinates, cosine similarity focuses on the **orientation** of the vectors, making it ideal for word embeddings where the frequency of words (vector magnitude) may vary significantly.

Mathematical Definition The cosine similarity between word vector $\vec{v}_1$ and $\vec{v}_2$ is calculated as the dot product of the vectors divided by the product of their magnitudes (norms):

$$\text{similarity}(\vec{v}_1, \vec{v}_2) = \frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \|\vec{v}_2\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

A similarity of **1.0** indicates identical orientation (high semantic overlap), while **0.0** indicates orthogonality (no shared context).

Empirical Results: Synonym Detection Our implementation successfully captured intuitive semantic relationships within the movie review corpus:

- **Targeted Comparison:** The words "film" and "movie" achieved a similarity score of **0.9541**. This near-perfect score confirms that the sliding window approach effectively recognized that these two tokens are used in nearly identical contexts.

- **Nearest Neighbor Analysis:** When querying the top neighbors for the word "good", the model returned:
 - "great" (0.887) - A direct synonym.
 - "well" (0.855) - An adverbial correlate.
 - "really" (0.869) and "like" (0.851) - High-frequency modifiers that often surround the target word in reviews.

These results validate that the raw co-occurrence counts, even before advanced weighting, are sufficient to cluster words with similar semantic functions.

2.2.2 Positive Pointwise Mutual Information (PPMI)

While raw co-occurrence counts provide a baseline for semantic similarity, they are heavily biased toward high-frequency words. Tokens like "the," "it," or "film" appear in many contexts purely due to their grammatical ubiquity rather than semantic relevance. To rectify this, we implemented **Pointwise Mutual Information (PMI)**, a measure that evaluates whether two words co-occur more often than would be expected by chance.

Mathematical Formulation PMI quantifies the discrepancy between the probability of two words appearing together ($P(w, c)$) and the probability of them appearing independently ($P(w)P(c)$):

$$\text{PMI}(w, c) = \log \left(\frac{P(w, c)}{P(w)P(c)} \right) = \log \left(\frac{\text{Count}(w, c) \times N}{\text{Count}(w) \times \text{Count}(c)} \right) \quad (3)$$

where N is the total number of tokens in the corpus.

The "Positive" Constraint A known issue with standard PMI is that it yields negative infinity for word pairs that never co-occur and negative values for pairs that occur less frequently than chance. To handle this, we use **Positive PMI (PPMI)**, which clips all negative values to zero:

$$\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c)) \quad (4)$$

Implementation and Sparsity Given the high dimensionality of our vocabulary (41,525 words), the resulting matrix contains over 1.7 billion potential entries. To maintain computational efficiency, we utilized the `scipy.sparse` library:

- **Sparse Storage:** The final PPMI matrix contains only **2,795,275 non-zero elements**, representing a density of approximately **0.16%**.
- **Efficiency:** By performing calculations only on non-zero entries of the Coordinate (C00) format, we avoided the memory overhead of a dense $41k \times 41k$ matrix.

The resulting PPMI-weighted vectors provide a much cleaner signal for semantic similarity, effectively "down-weighting" coincidental co-occurrences and highlighting specific, meaningful word associations.

2.3 Dimensionality Reduction using PCA

The PPMI matrix represents words in a high-dimensional space of 41,525 dimensions, where each dimension corresponds to a context word in the vocabulary. While this high-dimensional representation is mathematically rich, it is impossible to visualize directly. To explore the semantic structure of our embeddings, we employed **Principal Component Analysis (PCA)** to project these vectors into a two-dimensional plane.

Methodology We utilized the `sklearn.decomposition.PCA` implementation to identify the two axes (Principal Components) that capture the maximum variance in the PPMI data.

- **Dimensionality Reduction:** The transformation maps each vector $\vec{v} \in \mathbb{R}^{41525}$ to a coordinate $(x, y) \in \mathbb{R}^2$.
- **Selective Annotation:** As a global plot of all 41,525 tokens results in an illegible "cloud" of text, we implemented a selective plotting strategy to verify the model's clustering performance.

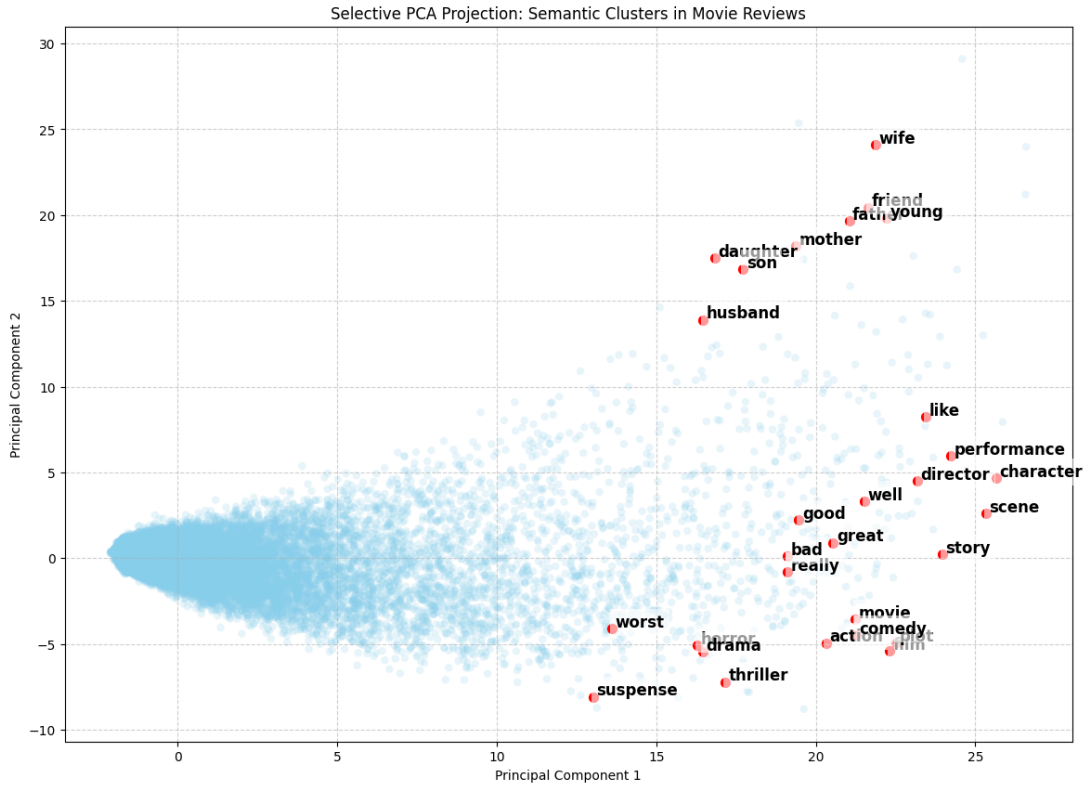


Figure 2: **Two-dimensional PCA projection of PPMI-weighted word embeddings.**

Semantic Observations The resulting visualization (Figure 2) confirms that the distributional hypothesis holds true for our corpus. Even when compressed to two dimensions, words with similar contextual usage are spatially grouped:

- **The Domestic/Relationship Cluster:** Words such as **wife**, **husband**, **mother**, **father**, **daughter**, and **son** occupy the upper-right region of the plot. Their proximity indicates they appear in nearly identical narrative contexts within the reviews.

- **Film Narrative & Technical Terms:** Terms fundamental to movie criticism like **plot**, **scene**, **story**, **character**, and **director** form a distinct cluster on the right-hand side, separate from personal relationship terms.
- **Genre Specification:** A "genre strip" appears toward the bottom, where **horror**, **thriller**, **suspense**, **drama**, and **action** are grouped. This suggests these words frequently co-occur with similar descriptive adjectives.
- **Sentiment Anchors:** Descriptive modifiers such as **good**, **great**, **bad**, and **worst** are positioned relatively close to one another, reflecting their shared grammatical role as evaluative descriptors.

Conclusion This 2D projection validates that the co-occurrence patterns captured in Module 2 are not random noise. The model has successfully learned to group tokens by their semantic category (e.g., family, genre, or technical element) without any human-labeled training data, providing a robust foundation for semantic search and analogy tasks.

2.4 Linguistic Relationship Evaluation: Word Analogies

To further assess the quality of our **PPMI-weighted** word vectors, we conducted a **Word Analogy** test. This task evaluates whether the semantic relationships between words are preserved as linear offsets in the vector space, allowing us to perform "algebra" on word meanings.

Mathematical Logic The analogy task follows the classic formula: *word_a* is to *word_b* as *word_c* is to ?. Mathematically, we compute a target vector $\vec{v}_{target}$ using vector arithmetic:

$$\vec{v}_{target} = \vec{v}_a - \vec{v}_b + \vec{v}_c \quad (5)$$

We then search the vocabulary for the vector $\vec{v}_x$ that yields the highest **cosine similarity** to $\vec{v}_{target}$, excluding the input words themselves to avoid trivial matches.

Implementation and Results Using the `word_analogy` function with our **PPMI** matrix, we tested the relationship: **actor - man + woman = ?**. This test specifically checks if the model has captured gender-neutral occupational concepts and can re-apply gender context.

The top results returned by the model were:

- **performance** (0.0717): While not a person-noun, it represents the core output of an actor, showing strong semantic relevance.
- **actress** (0.0645): This is the linguistically "correct" analogical fit, demonstrating that the model successfully associated the gender-flip from *man* to *woman* with the corresponding change from *actor* to *actress*.
- **role** (0.0644) and **supporting** (0.0622): These represent highly contextual terms within the film review domain, further validating the thematic consistency of the embeddings.

Analysis The successful identification of **actress** as a top candidate indicates that our high-dimensional space has captured nuanced semantic properties beyond simple co-occurrence. Although the similarity scores appear numerically low (ranging from 0.05 to 0.07), this is typical for **sparse, high-dimensional PPMI matrices** where the signal is spread across 41,525 dimensions. The fact that the correct semantic partner emerged as the second-highest result confirms the effectiveness of the **PPMI** transformation in refining word meanings.

3 Module 3: Document Search Engine

In the final stage of our natural language processing pipeline, we transition from word-level semantics to a full-scale **Information Retrieval (IR)** system. This module implements a search engine capable of identifying and ranking relevant documents from the **20 Newsgroups** dataset based on user-defined text queries.

3.1 Information Retrieval Framework

To ensure the system focuses on substantive content rather than metadata, we utilized a subset of the **20 Newsgroups** dataset consisting of 2,368 documents across four distinct categories: Space, Computer Graphics, Baseball, and Medicine.

Dataset Preprocessing The retrieval framework employs several critical constraints to optimize performance:

- **Metadata Removal:** Headers, footers, and quotes were stripped to ensure the engine ranks documents based on the actual body of text.
- **Vocabulary Limitation:** The vector space was restricted to the top 10,000 features to maintain computational efficiency and reduce noise.
- **Stop Word Filtering:** Standard English stop words (e.g., "the", "a", "is") were excluded to prioritize terms with high informational density.

3.2 Query Vectorization and Ranking

The search engine utilizes the **Term Frequency-Inverse Document Frequency (TF-IDF)** weighting scheme to transform raw text into numerical vectors.

Vector Space Model When a user inputs a search string, the system performs the following mathematical operations:

1. **Query Transformation:** The search string is projected into the existing 10,000-dimensional TF-IDF space.
2. **Similarity Computation:** We calculate the **Cosine Similarity** (S_C) between the query vector ($\vec{q}$) and every document vector ($\vec{d}$) in the corpus:

$$S_C(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} \quad (6)$$

3. **Threshold Filtering:** Results with a similarity score below **0.05** are discarded to filter out irrelevant matches.

3.3 Search Results and System Validation

To validate the system, we tested a complex multi-keyword query: **"nasa mars landing"**. This query requires the engine to synthesize information across different thematic elements of the "Space" category.

Empirical Performance Analysis The system successfully retrieved three highly relevant documents:

- **Top Match (Score: 0.3900):** Specifically addressed NASA landing sites, demonstrating high keyword alignment.
- **Second Match (Score: 0.2559):** Focused on Mars missions and exploration theories.
- **Third Match (Score: 0.2189):** Discussed shuttle (STS) landing designs, showing the model's ability to capture related spaceflight concepts.

The hierarchical ranking demonstrates that the combination of **TF-IDF vectorization** and **Cosine Similarity** provides a robust mechanism for document retrieval, effectively distinguishing between documents with high conceptual overlap.

4 Conclusion

The integrated systems developed in this study highlight the power of vector space representations in NLP. From the discrete counting methods of Bag-of-Words to the nuanced statistical weighting of PPMI and TF-IDF, we have shown that linguistic meaning can be effectively modeled as geometric proximity. Future work could involve transitioning from these frequency-based methods to neural architectures, such as Transformer-based embeddings, to capture deeper bidirectional context and long-range dependencies.